10 Essential JavaScript Notes You Need to Master

Quick, visual cheat-sheet for busy devs. Swipe through for concise, practical reminders and examples you'll actually use.

1. Variables: var, let, const

var

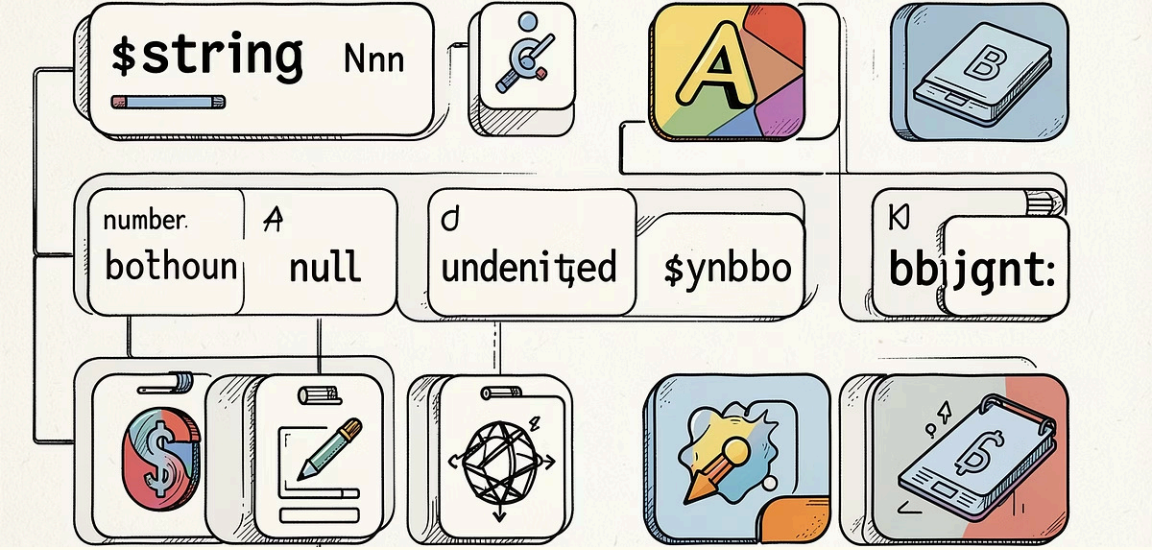
Function-scoped, hoisted. Avoid for predictable scope.

let

Block-scoped, reassignable. Use for mutable values.

const

Block-scoped, not reassignable. Use for constants and stable bindings.



2. Data Types: Primitives vs Objects

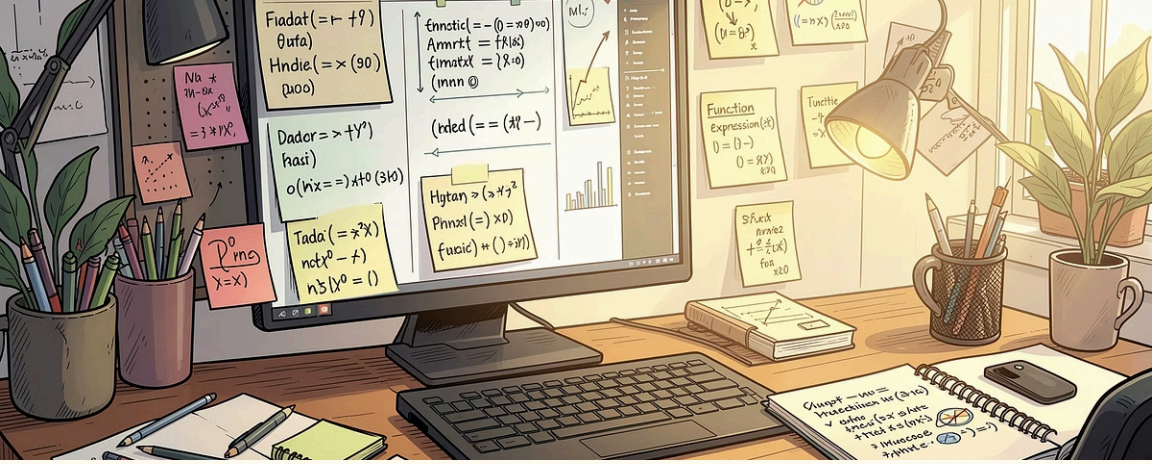
Primitives

String, Number, Boolean, Null, Undefined, Symbol, BigInt — immutable values.

Objects

Collections, mutable. Arrays and functions are objects too.

Remember: typeof null === "object" is a historical quirk.



3. Functions: Declare, Express, Arrow

Declaration

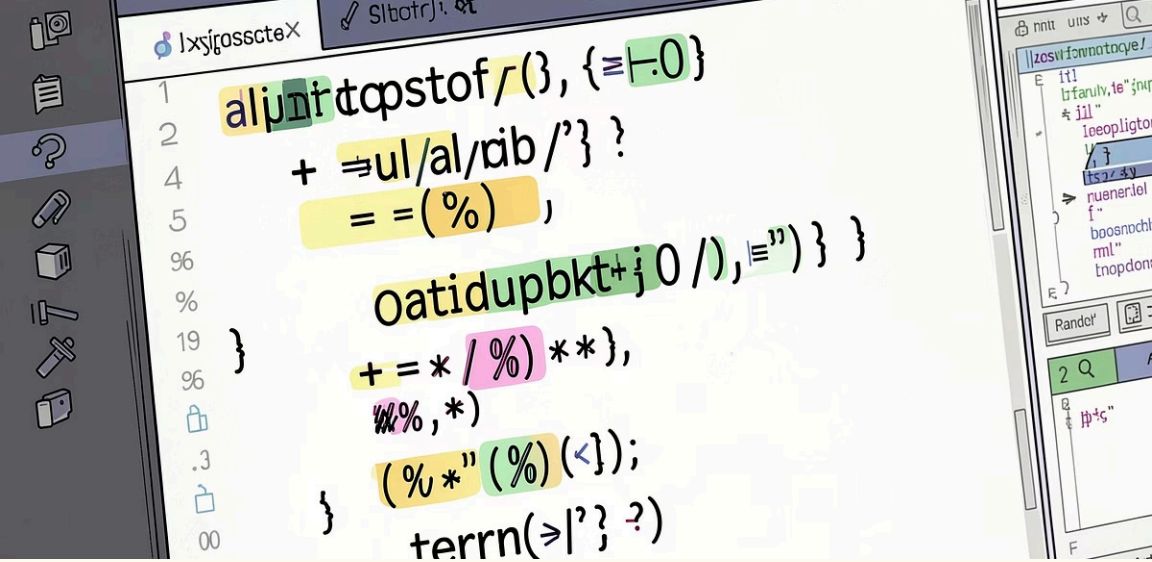
Hoisted: function greet() { return 'hi' }

Expression

Assignable: const f = function(){}

Arrow

Shorter syntax, lexical this: const f = () => {}



4. Operators: Quick Reference

- Arithmetic

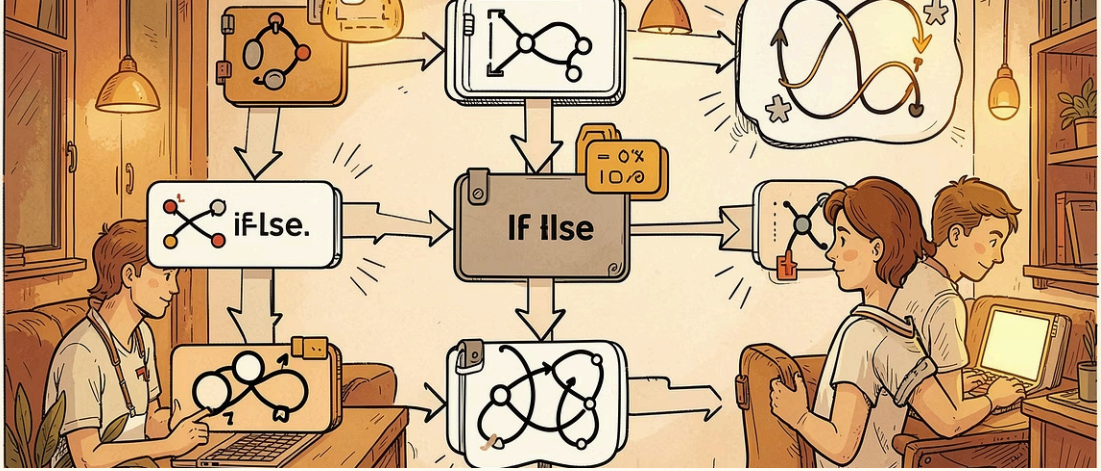
  + - * / % (mod), ** (power)

- Comparison & Logical

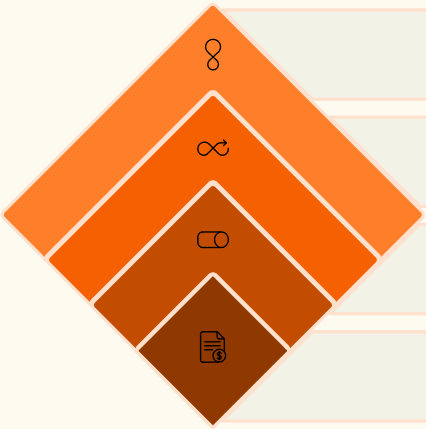
  == vs ===, &&, ||, ! — prefer === and !== for safety

- Ternary

  condition ? expr1 : expr2 — concise inline branch



5. Control Flow: Branches & Loops



While / Do
Condition-driven iteration

For Loop
Iterate with known range

Switch
Handle multiple cases

If / Else
Simple binary decision

Use if/else for simple decisions, switch for multiple discrete cases, and loops for iteration. Prefer for-of and array methods when possible.

6. Arrays: Key Methods

.map()

Transforms each item. Returns new array.

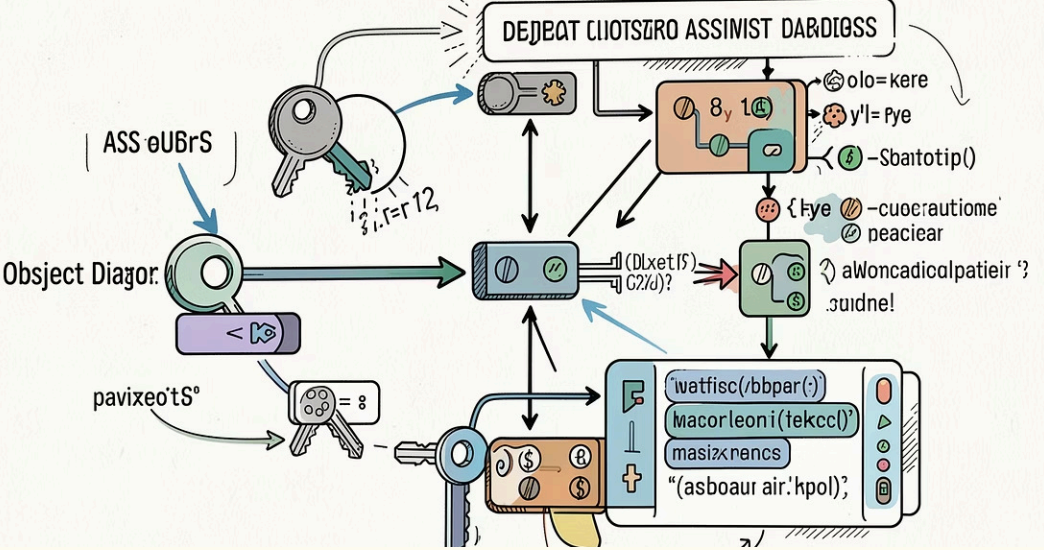
.filter()

Keeps items that match condition.

.reduce()

Accumulates values into single result.

Also use .forEach() for side effects; prefer non-mutating methods for clarity.



7. Objects: Access & Destructure

Access

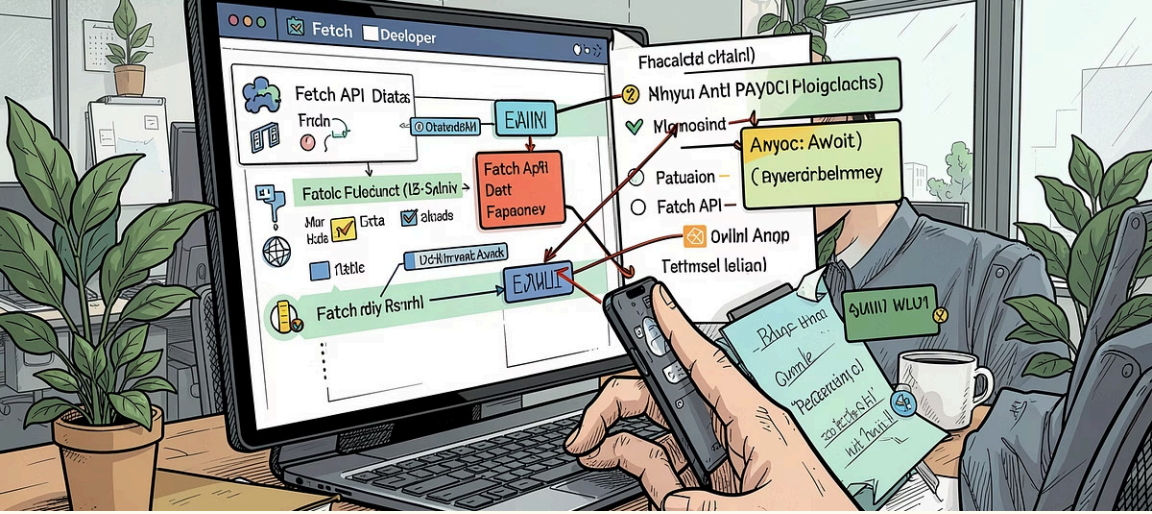
dot (obj.key) or bracket (obj['key']) notation for dynamic keys.

Destructuring

const {a, b} = obj — extracts properties into variables.

Methods

Define functions on objects: obj.method = function(){}



8. Asynchronous JavaScript

Promises

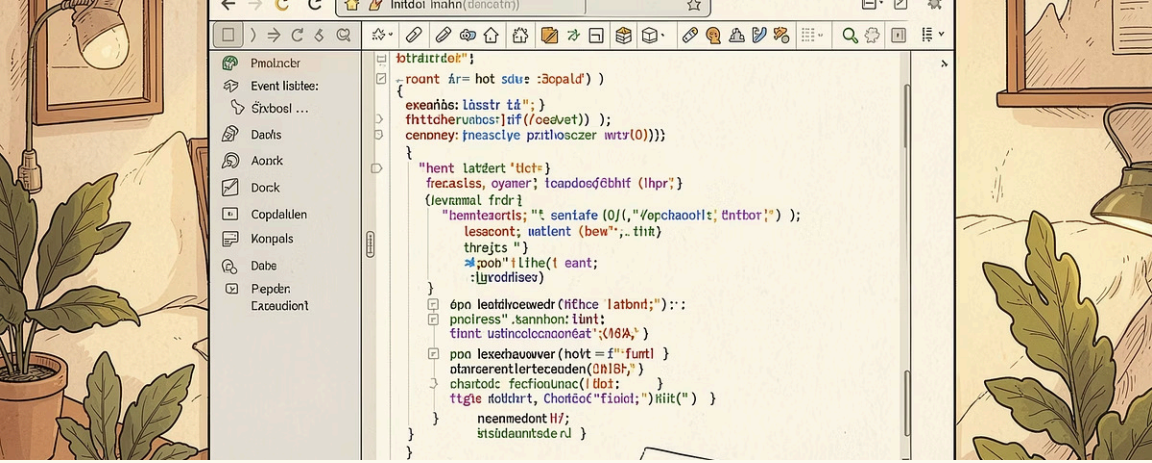
then/catch chain for async results.

async/await

Syntactic sugar for promises — write async code like sync.

Network

Use fetch or axios, handle errors and timeouts.



9. DOM Manipulation & Events

Select

document.querySelector / querySelectorAll to find elements.

Update

element.textContent, innerHTML, classList to change UI.

Events

addEventListener('click', fn) to react to user actions.

Prefer event delegation for many dynamic elements.